

DSA Strings

Complete Pattern Notes
Self-Study & Last-Minute Revision Guide

String Patterns — Overview

Pattern	Core Idea	When to Use
Sliding Window	Maintain window, track char frequency	Substring/window condition problems
Character Frequency	Compare freq maps of two strings	Anagram, grouping, permutation checks
KMP / Z-Algorithm	Reuse partial match info via LPS array	Pattern search, repeated substring, rotation
Palindrome	Expand around center / DP / Manacher's	Palindrome substring, count, partition
Rolling Hash	Convert window to number, $O(1)$ compare	Duplicate substring detection, large constraints

Other patterns used on strings (Two Pointer, DP, Stack, Trie, Backtracking) are general patterns covered in their own topics — not string-specific.

Pattern 1 — Sliding Window on Strings

Core Idea: Same template as array sliding window — but window state is a character frequency map, not a sum.

Array vs String Sliding Window

	Array Sliding Window	String Sliding Window
Window state	Integer sum	Character frequency map
Condition	sum >= target	freq matches, distinct count, repeats
Shrink trigger	sum exceeds limit	freq condition violated
Data structure	int variable	unordered_map or int[26]

The 3 Flavors

- Flavor 1 — Fixed window, match frequency: find all anagrams, permutation exists
- Flavor 2 — Variable window, optimize length: longest substring with condition
- Flavor 3 — Exactly K pattern: exactly K distinct = $\text{atMost}(K) - \text{atMost}(K-1)$

The formed Counter Trick

```
int required = pattern_map.size(); // distinct chars needed
int formed = 0; // distinct chars currently satisfied
// when adding char c:
window[c]++;
if(pattern.count(c) && window[c]==pattern[c]) formed++;
// window valid when: formed == required <- O(1) check, not O(k)
```

int[26] vs unordered_map: use int[26] for lowercase-only strings (faster, no collisions). Use unordered_map for mixed/unicode characters.

Key Problem — Longest Repeating Character Replacement

Key insight: Window valid when $(\text{window size}) - (\text{max frequency in window}) \leq K$. That difference is exactly how many characters need replacing.

```
while((right-left+1) - maxFreq > k){
    freq[s[left]-'A']--;
    left++;
}
```

maxFreq never decreases even when its character leaves the window — safe because a smaller maxFreq only gives a smaller/equal window, which we don't care about (we track the max already found).

Key Problem — At Least K Repeating Characters

Why standard sliding window breaks: Removing a character from the left doesn't fix the violation if the offending character is in the middle of the window. No monotonic property.

The fix — Divide and Conquer: Characters appearing less than K times can never be part of the answer. Split the string at them and solve each piece independently.

```
for each character c with freq[c] < k:
    split string at c
    recursively solve left part and right part
    return max(left, right)
if no character has freq < k: return s.length() // base case, avoids infinite recursion
```

Critical bug to avoid: always have a base case that returns immediately when the whole string is already valid — otherwise infinite recursion on the unsplit string.

Solved Problems

Problem	LC#	Key Insight
Find All Anagrams in a String	438	Fixed window + freq vector comparison
Longest Repeating Character Replacement	424	windowSize - maxFreq <= k
Longest Substring with At Least K Repeating	395	Divide and conquer at chars with freq < k

Pattern 2 — Character Frequency / Anagrams

Core Idea: Two strings are anagrams if and only if they have identical character frequency maps.

The 3 Core Operations

1. Build frequency map

```
int freq[26] = {0};  
for(char c : s) freq[c-'a']++;
```

2. Compare two maps — single diff trick

```
int freq[26] = {0};  
for(char c : s) freq[c-'a']++; // add for s  
for(char c : t) freq[c-'a']--; // subtract for t  
// anagrams if all entries are 0
```

3. Sliding update (for window-based anagram problems)

```
freq[s[right]-'a']++; // incoming  
freq[s[left]-'a']--; // outgoing
```

The 4 Problem Types

Type	Example	Approach
Check if anagrams	"cat" vs "tac"	Single freq map, add/subtract, check zeros
Find all anagram positions	find "ab" in "cbaebabacd"	Sliding window + freq comparison
Group anagrams together	group ["eat","tea","ate"]	Sorted string or freq string as hashmap key
Min deletions to anagram	count chars to delete	Count only positive mismatches in freq diff

The Frequency Map as Key Trick

Most powerful technique in this pattern — any two strings with the same freq map ARE anagrams, so the freq map is the canonical form of an anagram group.

	Sort Key	Freq Key
Time	$O(k \log k)$ per string	$O(k)$ per string
Readability	Cleaner	More verbose
Use when	k is small	k is large / performance matters

Solved Problems

Problem	LC#	Key Insight
Valid Anagram	242	Single freq map, process both strings in one loop

Problem	LC#	Key Insight
Group Anagrams	49	Sorted string as hashmap key
Minimum Steps to Make Two Strings Anagram	1347	Only sum positive freq diffs — negatives cancel out

Pattern 3 — KMP Algorithm

*Core Idea: Reuse information from a partial match instead of restarting from scratch on mismatch. Runs in $O(n+m)$ vs $O(n*m)$ brute force.*

The LPS Array — Most Important Concept

LPS = Longest Proper Prefix which is also Suffix. $\text{lps}[i]$ = length of longest proper prefix of $\text{pattern}[0..i]$ that is also a suffix of it.

```
pattern = "aabcaab"
lps     = [0, 1, 0, 0, 1, 2, 3]

What lps[i] tells you: on mismatch at pattern[i+1],
jump to pattern[lps[i]] instead of pattern[0].
```

Building the LPS Array

```
vector<int> buildLPS(string pattern){
    int m = pattern.size();
    vector<int> lps(m, 0);
    int len = 0;    // length of previous longest prefix suffix
    int i = 1;
    while(i < m){
        if(pattern[i] == pattern[len]){
            len++; lps[i] = len; i++;
        } else {
            if(len != 0) len = lps[len-1];    // fall back, don't increment i
            else { lps[i] = 0; i++; }
        }
    }
    return lps;
}
```

Key line: $\text{len} = \text{lps}[\text{len}-1]$. On mismatch with $\text{len} > 0$, fall back to the previous longest prefix-suffix instead of giving up entirely — same idea used in the search itself.

The KMP Search

```
int i=0, j=0;    // i for text, j for pattern
while(i < n){
    if(text[i]==pattern[j]){ i++; j++; }
    if(j == m){
        record match at i-j;
        j = lps[j-1];    // look for next match
    } else if(i<n && text[i]!=pattern[j]){
        if(j != 0) j = lps[j-1];    // use LPS to skip
        else i++;    // no fallback possible
    }
}
```

```
}
}
```

Useful LPS Tricks

Use Case	Formula / Approach
Check if s2 is rotation of s1	Check if s1 is substring of (s2+s2)
Shortest repeating period	$\text{period} = m - \text{lps}[m-1]$
Repeated Substring Pattern check	if $m \% \text{period} == 0$ AND $\text{period} != m \rightarrow \text{true}$

Pattern Matching Algorithms Compared

Algorithm	Time	Best For
Brute Force	$O(n*m)$	Very short patterns
KMP	$O(n+m)$	Single pattern, guaranteed linear
Z-algorithm	$O(n+m)$	Same as KMP, prefix-focused style
Rabin-Karp	$O(n+m)$ avg	Multiple pattern search, rolling hash
Aho-Corasick	$O(n+m+z)$	Many patterns simultaneously

Interview reality: `s.find(pattern)` is good enough for most problems. KMP is asked when $O(n+m)$ is explicitly required, when you need ALL occurrences, or when the LPS array properties matter (rotation/period problems).

Solved Problems

Problem	LC#	Key Insight
Find Index of First Occurrence in String	28	Standard KMP search using LPS array
Repeated Substring Pattern	459	$\text{period} = m - \text{lps}[m-1]$, check $m \% \text{period} == 0$ and $\text{period} != m$

Pattern 4 — Palindrome

Core Idea: A palindrome is symmetric around its center. Every palindrome problem reduces to finding or verifying that center.

The 2 Types of Centers

Odd length palindrome: "aba" -> center is a single character 'b'

Even length palindrome: "abba" -> center is between two characters

A string of length n has $2n-1$ possible centers.

Expand Around Center — Template

```
int expandFromCenter(string s, int left, int right){
    while(left >= 0 && right < s.size() && s[left] == s[right]){
        left--; right++;
    }
    return right - left - 1;    // length of palindrome
}
```

Why right-left-1: when loop exits, left and right have both gone one step too far past the mismatch. Actual palindrome length is $(right-1)-(left+1)+1 = right-left-1$.

Longest Palindromic Substring — Full Approach

```
for each index i:
    len1 = expandFromCenter(s, i, i)    // odd length center
    len2 = expandFromCenter(s, i, i+1)  // even length center
    track the longest, with start = i - (len-1)/2
```

Time complexity: $O(n^2)$ — n centers, each expansion $O(n)$ worst case.

Palindrome with DP

Use when you need to query 'is $s[i..j]$ palindrome' many times repeatedly, or combine with another DP (like palindrome partitioning).

```
isPalin[i][i] = true;    // base case
for len = 2 to n:
    for each i with j = i+len-1:
        if len==2: isPalin[i][j] = (s[i]==s[j])
        else: isPalin[i][j] = (s[i]==s[j]) && isPalin[i+1][j-1]
```

Manacher's Algorithm — $O(n)$

Core insight: If you know palindrome info about earlier centers, skip redundant expansion for later centers using symmetry (mirror index).

```
transform: "abc" -> "^#a#b#c#$"    // unifies odd/even, adds sentinels
P[i] = min(right - i, P[mirror])    // reuse info if i is inside known palindrome
```


then try expanding further from there

When to Use Which

Situation	Use
Find longest palindrome, simple problem	Expand around center — $O(n^2)$, simple
Need $O(n)$ guaranteed (huge strings)	Manacher's
Query many i, j ranges repeatedly	DP table
Combined with another DP (partitioning)	DP table

Manacher's is rarely required to implement from scratch in interviews — $O(n^2)$ expand-around-center is usually accepted for Leetcode 5.

Solved Problems

Problem	LC#	Key Insight
Longest Palindromic Substring	5	Expand around center, both odd and even
Palindromic Substrings (Count)	647	Same expansion, count every successful step instead of max
Palindrome Partitioning II	132	Needs DP — revisit in DP topic

Pattern 5 — Rolling Hash (Rabin-Karp)

Core Idea: Convert each window into a number (hash). Update the hash in $O(1)$ when window slides instead of recomputing from scratch.

The Hash Formula

`s = "abc", base B`

`hash = a*(B2) + b*(B1) + c*(B0)`

Common values: B = 31 (lowercase letters), MOD = 1e9+7

Why B=31: must be larger than alphabet size and prime — reduces collision patterns. Not mathematically unique, just common convention.

The Rolling Update — Two Steps

`// Step 1: remove leftmost character's contribution`

`winHash = (winHash - (text[i]-'a'+1)*power % MOD + MOD) % MOD;`

`// Step 2: shift remaining left, add new rightmost character`

`winHash = (winHash * B + (text[i+m]-'a'+1)) % MOD;`

where `power = B(m-1) % MOD` (positional weight of leftmost char)

Why +MOD before final %MOD: subtraction can go negative since winHash might be smaller than the term being removed. Adding MOD guarantees a positive result before final modulo.

Numeric Walkthrough (base 10 for clarity)

window "123": `hash = 1*100 + 2*10 + 3 = 123`, power=100

Remove leftmost '1': `123 - 1*100 = 23`

Shift and add '4': `23*10 + 4 = 234` <- matches `hash("234")` exactly

Hash Collision — Critical Issue

Two different strings CAN produce the same hash. Always verify with actual string comparison when hashes match.

```
if(windowHash == patternHash){
    if(text.substr(i, m) == pattern) // verify before trusting
        recordMatch();
}
```

Double hashing: compute two independent hashes (different B, MOD pairs). Collision only if BOTH match — probability drops drastically.

One String vs Two Strings

Scenario	Approach
Pattern + text (two strings)	Compare window hash to one fixed pattern hash
Single string (find duplicates)	Store every window hash in a hashset, check if seen before

When Rolling Hash Is Actually Needed

Signal	Why
Massive input + substring comparisons	substr() copy is $O(k)$, too slow at scale
Comparing many substrings repeatedly	$O(1)$ hash compare beats $O(k)$ string compare
Binary search on string length	Needs $O(n)$ validity check at each length
Problem explicitly hints at hashing	Direct signal

The real trigger question: 'Will my simple substring/hashmap solution TLE given the constraints? If yes, replace string comparison with $O(1)$ hash comparison.' You don't start with rolling hash — you optimize INTO it.

Interview Reality Check

Context	Rolling Hash Needed?
Most FAANG interviews	Rarely required
Most company interviews	Almost never required
Competitive programming	Required
System design (conceptual)	Helps — consistent hashing concept

Recommendation: understand the concept, don't over-invest in perfecting implementation. Mentioning Rabin-Karp conceptually in an interview is often enough.

Solved Problems

Problem	LC#	Key Insight
Repeated DNA Sequences	187	Fixed window=10. Simple substr+hashmap works fine at given constraints; rolling hash is the scalable alternative

Quick Reference — All 5 String Patterns

Pattern	Time	Use When
Sliding Window	$O(n)$	Window-based character frequency condition
Character Frequency	$O(n)$	Anagram check, grouping, permutation match
KMP / Z-Algorithm	$O(n+m)$	Pattern search needing guaranteed linear time
Palindrome	$O(n^2)$ / $O(n)$	Palindromic substring, count, or partition
Rolling Hash	$O(n+m)$ avg	Duplicate substring detection at scale

Most Important Rules to Remember

- Sliding Window on strings: window state is a freq map, not a sum. Use formed counter for $O(1)$ validity checks.
- Character Frequency: anagrams = identical freq maps. Sorted string or freq string makes a clean hashmap key.
- KMP: LPS array tells you where to resume after a mismatch. $len = lps[len-1]$ is the fallback rule.
- Palindrome: always check BOTH odd-center and even-center expansions. $start = i - (len-1)/2$.
- Rolling Hash: always verify on hash match (collisions exist). Use power = $B^{(m-1)}$ to remove leftmost contribution correctly.
- Other string-applied patterns (Two Pointer, DP, Stack, Trie, Backtracking) are covered in their dedicated topics.

— End of Notes —